

Implementing the reducers

Create a file `contactReducers.ts` under the folder `reducers`. When the store dispatches an action, it passes to `rootReducer`, the state maintained in the store and the action. While creating the `rootReducer`, we would have combined all the individual reducers of the application using the `combineReducers` API. In our current application, we have only one reducer, i.e., for contacts, which we will be creating shortly. Although we have only one reducer, it will be useful to use the `combineReducers` API, as we can extend it to add more reducers as our application expands in the future.

```
const rootReducer = combineReducers({ contacts });
```

The `rootReducer` will pass the action and the respective state to each reducer. Let's then implement the reducer function `contactReducer` accepting two parameters — *state* and *action*. The reducer is supposed to handle the action it is interested in and return the new state. An important point to understand is that the reducer is a *pure function*, and it should not mutate the parameters and return a new state in an immutable way, using only the values passed in the parameters. In the `contactReducer`, we will handle the actions – successful loading of contacts, successful addition of a contact and successful deletion of a contact.

```
const contacts = (state, action) => {
  switch (action.type) {
    case 'LOADED_CONTACTS':
      // Return contacts

    case 'ADDED_CONTACT':
      // Add new contact to state and return

    case 'DELETED_CONTACT':
      // Delete contact from state and return

    default:
      // Return state passed as parameter
  }
}
```

Implementing the actions

Create a file `actionTypes.ts` under the folder `actions` to store all the action names as a constant. Having the action types as a constant will help in better maintenance of code rather than using them directly as a string.

Next, create a file `contactActions.ts`, in which we will implement various actions related to *Contacts*. We will implement the actions—loading of contacts, adding a contact and deleting a contact. As we will be communicating using REST API calls with a server in the backend, we will be making asynchronous calls and hence will use the *redux-thunk* middleware to perform asynchronous dispatch. The

thunk function will invoke the Contact Service methods (discussed below), which will communicate with the server and fetch or add/modify the data based on the action. The Contact Service makes REST API calls to the Express server, performs the necessary task and returns either success or error. If success is returned, then the corresponding success action—load, add or delete—will be dispatched, after which the reducer code discussed above will get executed.

```
export class ContactActions {
  loadContacts() {
    return dispatch => {
      // Invoke Contact Service method to load contacts and
      process
      // successful result or error
    }
  }
  addContact(contact) {
    return dispatch => {
      // Invoke Contact Service method to add contact and
      process
      // successful result or error
    }
  }
  deleteContact(id) {
    return dispatch => {
      // Invoke Contact Service method to delete contact and
      process
      // successful result or error
    }
  }
}
```

Implementing the Contact Service

To create the Contact Service, run the Angular CLI *generate service* command from the folder `'app'`.

```
ng generate service contact
```

The above command will generate the standard service blueprint containing the service typescript file and a service test specification file.

Let's implement the service methods – *getContacts*, *addContact* and *deleteContact*. To implement these methods, issue REST API calls to the Express server using the Angular HTTP module.

```
@Injectable()
export class ContactService {
  getContacts() {
    // Invoke REST endpoint '/api/contacts' using http.get
  }

  addContact(contact) {
```